DESIGN AND ANALYSIS OF ALGORITHMS

Practical Examination — Complete Revision & Viva Guide

Five assignment topics • Six Python programs

Fibonacci & Tower of Hanoi | Huffman Coding | Fractional Knapsack | 0/1 Knapsack (DP and Branch & Bound) | N-Queens

A single study companion covering problem statements, theory, algorithm steps, code structure, design choices, complexity, sample tests, and likely viva questions.

Use your local .py files as the final authority for exact prompts, variable names, and printed messages.

How to use this guide

There are five practical topics but six Python files because 0/1 Knapsack is implemented with two separate strategies. For each practical, learn: what the problem asks, the algorithmic idea, how the code implements it, why important design choices were made, and the complexity.

File	Purpose
Fibonacci_Hanoi.py	Recursive and iterative Fibonacci; recursive Tower of Hanoi
Huffman_Coding.py	Huffman tree construction, encoding and decoding using a min-heap
Fractional_Knapsack_Greedy.py	Fractional Knapsack using greedy ratio ordering
01Knapsack_DP.py	0/1 Knapsack using Dynamic Programming
01Knapsack_BranchNbound.py	0/1 Knapsack using Branch and Bound
NQueens.py	N-Queens backtracking with a fixed first queen

A strong viva answer structure

• Define the problem in one or two sentences.

• Name the technique: recursion, greedy, dynamic programming, Branch and Bound, or backtracking.

• Explain the key decision, recurrence, or data structure.

• Connect the idea to the important function in the code.

• State time and space complexity, explaining the variables used.

Core concepts

Technique	Meaning	Used in
Recursion	Solve smaller instances by calling the same function.	Fibonacci, Hanoi, Huffman traversal, B&B, N-Queens
Greedy	Choose a locally best option at each step.	Huffman, Fractional Knapsack
Dynamic Programming	Store and reuse solutions to overlapping subproblems.	0/1 Knapsack DP
Branch and Bound	Explore choices and prune branches using a bound.	0/1 Knapsack B&B
Backtracking	Try a choice; undo it if it cannot lead to a solution.	N-Queens

Time complexity describes how operations grow with input size. **Space complexity** describes memory use. Big-O is an asymptotic upper bound, not an exact stopwatch time. Auxiliary space means extra working memory, such as a recursion stack or DP table.

1. Fibonacci Sequence and Tower of Hanoi

File: Fibonacci_Hanoi.py

Problem statement

Generate the first n Fibonacci terms using recursive and non-recursive methods, compare measured execution times and theoretical complexities, and solve Tower of Hanoi for a given number of disks using recursion.

Fibonacci theory

The Fibonacci sequence begins 0, 1; every later term is the sum of the previous two:

```
F(0) = 0\nF(1) = 1\nF(n) = F(n-1) + F(n-2), for n >= 2
```

The recursive function follows the recurrence directly. It returns n for base cases 0 and 1; otherwise it calls itself for n–1 and n–2. The iterative function keeps two values, prints the current one, and updates them in a loop.

Code explanation and design choices

• **fib_recursive(n):** directly implements the mathematical recurrence. Base cases stop recursion.

• **fib_non_recursive(n):** uses two running variables. It avoids recalculating earlier terms and needs constant extra memory.

• **Timing:** time.perf_counter() is read before and after the sequence calculation; the difference is multiplied by 1,000,000 to display microseconds.

• **Input limits:** the discussed code limits Fibonacci n to 0–35 and Hanoi disks to 0–15 to keep recursive work and printed moves manageable.

• **Move counter:** a one-element list is passed through recursive Hanoi calls so all calls update the same mutable counter.

Why recursive Fibonacci is inefficient

The simple recursive version repeats work. For example, fib(5) and fib(4) both require some of the same smaller Fibonacci values. Iteration calculates each term once.

Tower of Hanoi theory

Move n disks from source to destination using an auxiliary peg. Move one disk at a time and never put a larger disk on a smaller disk. The algorithm moves n–1 disks to the auxiliary peg, moves the largest disk, then moves n–1 disks onto it.

```
Hanoi(n, source, auxiliary, destination):\n    if n == 0: return\n    Hanoi(n-1, source, destination, auxiliary)\
```

The recurrence is T(n)=2T(n–1)+1, T(0)=0. Therefore the exact number of moves is 2^n–1.

Complexity

Part	Time	Space	Reason
Recursive Fibonacci	O(2^n) upper bound	O(n)	Repeated calls; stack depth grows with n.
Iterative Fibonacci	O(n)	O(1)	One loop and fixed number of variables.
Tower of Hanoi	O(2^n)	O(n)	2^n–1 moves; recursion depth n.

More tightly, recursive Fibonacci grows as Θ(ϕ^n), where ϕ is the golden ratio. O(2^n) is still a valid commonly used upper bound for this simple implementation.

Sample checks

• 7 Fibonacci terms → 0 1 1 2 3 5 8.

• 3 Hanoi disks → 7 moves.

• 0 terms and 0 disks → no terms and zero moves.

• 1 disk → one move from source directly to destination.

Likely viva questions

Q. Why is a base case necessary?

A. It stops recursive calls; without it, calls continue until recursion/stack failure.

Q. Why is recursive Fibonacci slow?

A. It recalculates the same smaller Fibonacci values many times.

Q. Why is iterative Fibonacci O(1) space?

A. Only two running terms and a loop variable are needed, independent of n.

Q. What does perf_counter() do?

A. It provides a high-resolution clock for measuring elapsed time.

Q. Why do timings vary?

A. System load, interpreter overhead, CPU scheduling, and printing affect measured time.

Q. Does timing measure only computation?

A. If printing occurs inside the timed section, it includes output overhead too.

Q. Why is Hanoi T(n)=2T(n−1)+1?

A. There are two transfers of n−1 disks and one move of the largest disk.

Q. Why use a list as move_count?

A. Integers are immutable; a shared mutable list lets recursive calls update the same counter.

Q. What is the minimum number of Hanoi moves?

A. 2^n−1.

Q. What is the auxiliary peg for?

A. It temporarily holds smaller disks while the largest disk is moved.

2. Huffman Coding

File: Huffman_Coding.py

Problem statement

Construct Huffman codes for characters in a text, encode the text into bits, and decode a bit string back into text.

Theory

Huffman Coding is a lossless compression technique. Frequent characters generally receive shorter codes and less frequent characters longer codes. The codes are prefix-free: no complete character code is the prefix of another. This makes decoding unambiguous. Construction is greedy: repeatedly combine the two nodes with the smallest frequencies.

How this exact Python design works

• **Frequency dictionary:** freq[ch] counts how often each character appears. freq.get(ch, 0)+1 increments its count.

• **Heap entries:** (frequency, count, node). heapq is Python's min-heap; the smallest tuple is popped first.

• **Why count is included:** equal frequencies need a tie-breaker. A unique count avoids Python comparing unlike node values, such as a string leaf and an internal tuple.

• **Internal node:** represented as (left, right). A leaf is represented directly by a character string.

• **Tree building:** pop the two smallest nodes, make them left/right children of a new node, and push the combined frequency back. Repeat until one root remains.

• **Code generation:** left adds 0, right adds 1. At a leaf, save the path in the codes dictionary.

• **Encoding:** replace each character by its code and concatenate the bits.

• **Decoding:** follow left for 0 and right for 1. When a leaf is reached, append its character and reset to the root.

Single-character special case

If the text contains only one distinct character, the tree is just one leaf and has no edges. The code assigns that character code 0. The decoder has a matching special case: a string of zero bits represents the character repeated that many times.

Validation behavior

• A bit other than 0 or 1 is rejected.

• If decoding ends partway down the tree rather than at the root, the last codeword is incomplete and rejected.

• For a one-character tree, any bit other than 0 is rejected.

• Encoding a character absent from the built tree returns None and the menu reports that it is not found.

• The menu asks the user to build a tree before encoding or decoding.

Complexity

Operation	Complexity	Meaning
Count frequencies	O(L)	Scan L characters.
Build Huffman tree	O(k log k)	Heap operations for k distinct characters.
Generate codes	O(k)	Traverse a tree with O(k) nodes.
Encode	O(L+B)	Read L characters and produce B bits.
Decode	O(B)	Process each of B bits once.

Exact codewords can differ when frequencies tie. Different tie-breaking may produce different but valid trees. The reliable test is to encode and decode using the same tree; decoded text must equal original text.

Likely viva questions

Q. Is Huffman lossless?

A. Yes. With the same tree, decoding reconstructs the original text.

Q. Why do frequent characters get shorter codes?

A. This reduces the total number of bits needed for the input.

Q. What is prefix-free?

A. No codeword is the prefix of another complete codeword.

Q. Why use a min-heap?

A. It efficiently returns the two smallest frequencies at every merge.

Q. Why combine the two smallest frequencies?

A. That is the greedy choice in Huffman's optimal prefix-code construction.

Q. What do 0 and 1 mean?

A. 0 means go left; 1 means go right.

Q. Why reset to root after a decoded character?

A. The next codeword starts at the root.

Q. Why include count in the heap tuple?

A. It breaks frequency ties and avoids comparing unlike node objects.

Q. Can two characters have the same code?

A. No; distinct leaves have distinct paths in the tree.

Q. Will equal frequencies always give identical codes?

A. No. Tie-breaking may change the tree, but codes remain valid.

Q. Why reject an incomplete bit sequence?

A. It ends in the middle of a path and does not represent a complete character.

Q. What if decoding uses a different tree?

A. The result may be incorrect or rejected; encoder and decoder must share the tree.

3. Fractional Knapsack — Greedy Method

File: Fractional_Knapsack_Greedy.py

Problem statement

Given items with values and weights and a knapsack capacity, maximize total value. Fractions of items are allowed.

Theory and greedy choice

Calculate each item's value density, value/weight, and sort in descending order. Take as much as possible from the highest-density item first. If the next item does not fit completely, take the fraction that fills the remaining capacity.

```
ratio = value / weight\nfraction = remaining_capacity / item.weight\nadded_value = item.value * fraction
```

Code explanation

• The Item class stores id, value, weight, and precomputed ratio.

• items.sort(..., reverse=True) sorts from highest ratio to lowest.

• remaining_capacity begins at full capacity and decreases after each selection.

• If item.weight <= remaining_capacity, take the whole item and add its full value.

• Otherwise, calculate the fraction that fits, add proportional value, set remaining capacity to zero, and stop.

• The function returns total_value; main prints the result.

Worked example

Item	Value	Weight	Ratio
1	60	10	6
2	100	20	5
3	120	30	4

Capacity = 50. Take item 1 fully (10 weight, 60 value), item 2 fully (20 weight, 100 value), and 20/30 = 2/3 of item 3 (80 value). Total = 60+100+80 = **240**.

Why greedy is optimal here

Items are divisible. If some weight is assigned to a lower-ratio item while a higher-ratio item remains available, moving that weight to the higher-ratio item cannot reduce value. This exchange reasoning supports the greedy strategy. It does not automatically apply to 0/1 Knapsack, where items cannot be split.

Complexity and code-specific notes

Sorting is O(n log n); the selection loop is O(n); total time is O(n log n). The item list uses O(n) space. The discussed original version skips non-positive item weights and stops if no valid items remain. It rejects capacity <= 0, although mathematically zero capacity could return zero. It does not explicitly reject negative profits or catch every non-numeric input.

Likely viva questions

Q. What is the objective?

A. Maximize total value without exceeding capacity, allowing fractions.

Q. Why sort by value/weight?

A. It measures value gained per unit of capacity.

Q. Why does greedy work?

A. Divisibility lets us replace lower-density weight with higher-density weight without reducing value.

Q. What does fraction calculate?

A. The proportion of the current item that fits in remaining capacity.

Q. Why stop after taking a fraction?

A. That fraction fills the remaining capacity.

Q. What if all items fit?

A. All are taken fully and their values are added.

Q. What if capacity exceeds total item weight?

A. All items are taken; capacity remains unused.

Q. What if ratios tie?

A. Either order gives the same value per unit weight; optimal total is unaffected.

Q. What is the time complexity?

A. O(n log n), dominated by sorting.

Q. Why is this different from 0/1 Knapsack?

A. Fractional permits part of an item; 0/1 requires whole-item decisions.

4. 0/1 Knapsack — Two Implementations

Files: 01Knapsack_DP.py and 01Knapsack_BranchNbound.py

Common problem statement

Given n items, each with a value and weight, and capacity W, select a subset maximizing total value without exceeding W. Every item is either selected completely (1) or not selected (0).

Why not simply use Fractional Knapsack's greedy method?

A high value/weight ratio does not guarantee the best whole-item combination. One attractive item may consume capacity that would allow a better combination of other whole items. The repository therefore includes two exact strategies.

Shared worked example

Item	Value	Weight
1	60	10
2	100	20
3	120	30

Capacity = 50. The optimum is items 2 and 3: weight 20+30=50, value 100+120=**220**. Both implementations should find maximum profit 220.

A. Dynamic Programming — 01Knapsack_DP.py

The state dp[i][w] means the maximum value obtainable using the first i items with capacity w. For each item, compare excluding it with including it if it fits:

```
if weights[i-1] <= w:\n    dp[i][w] = max(\n        values[i-1] + dp[i-1][w-weights[i-1]],\n        dp[i-1][w]\n
```

The first row (zero items) and first column (zero capacity) are zero. The program fills a table of (n+1) rows and (W+1) columns, prints it, then traces backward to identify selected items. If dp[i][w] differs from dp[i–1][w], item i is selected and its weight is subtracted from w. If equal, the item is treated as excluded; this is valid even when an optimum tie exists.

DP complexity

Resource	Complexity	Reason
Time	O(nW)	There are O(nW) table cells, each filled in constant time.
Space	O(nW)	The full 2D table is stored.

This is pseudo-polynomial: runtime depends on numeric capacity W, not only on the number of bits used to represent W.

B. Branch and Bound — 01Knapsack_BranchNbound.py

The algorithm explores include/exclude decisions in a recursive search tree. It keeps the best valid profit found so far (the incumbent). An upper bound estimates the best profit a branch could possibly achieve. If that bound is no greater than the incumbent, the branch is pruned.

• Items are sorted by descending value/weight ratio to make the bound useful.

• The recursive state tracks next item index, current weight, current profit, and selected ids.

• When current profit improves the best profit, the code saves a copy of selected ids.

• The include branch is explored only if the item fits; after recursion, append/pop restores the working selection.

• The exclude branch explores the next item without adding its weight or value.

• The bound greedily adds whole remaining items and then estimates a fraction of the next item.

Why a fractional estimate is safe

Allowing a fraction relaxes the 0/1 restriction, so the fractional solution is at least as optimistic as the best whole-item solution in that branch. If even this upper bound cannot beat the incumbent, no valid 0/1 selection below that branch can improve the answer. Fractions are used only for the bound; final selected items remain whole.

The nested recursive function updates best_profit and best_items from its enclosing function. Python's nonlocal declaration means assignments refer to those enclosing variables. selected.copy() saves a snapshot so later append/pop operations do not mutate the saved best selection.

B&B; complexity

Sorting takes O(n log n). The search has exponential worst-case behavior because it may explore up to 2^n include/exclude combinations. Since bound calculation can scan remaining items at a node, a conservative worst-case time bound for this implementation is O(n·2^n). Recursion depth is O(n), in addition to item storage.

DP vs B&B;

Aspect	DP	Branch and Bound
Structure	2D table	Recursive decision tree
Avoids work by	Reusing subproblem answers	Pruning branches using upper bound
Worst-case time	O(nW)	Exponential; conservatively O(n·2^n) here
Space	O(nW) table	O(n) recursion depth plus item/selection storage
Good fit	Capacity W manageable	Bounds prune many branches

Likely viva questions

Q. What does 0/1 mean?

A. Each item is taken whole or not taken.

Q. What does dp[i][w] mean?

A. Best value using the first i items with capacity w.

Q. Why use the previous DP row?

A. It prevents reusing the same item and enforces the 0/1 rule.

Q. Why initialize row and column to zero?

A. Zero items or zero capacity gives zero value for standard non-negative inputs.

Q. Why O(nW)?

A. There are n×W-scale states, each computed in constant time.

Q. Why pseudo-polynomial?

A. Complexity depends on numeric capacity W.

Q. How does DP backtracking find items?

A. Compare adjacent rows; a changed value indicates the current item contributed to the optimum.

Q. What is branching?

A. Creating include and exclude choices for the next item.

Q. What is a bound?

A. An optimistic estimate of the best result reachable from a search state.

Q. What is pruning?

A. Discarding a branch whose bound cannot beat the incumbent.

Q. What is the incumbent?

A. The best valid solution found so far.

Q. Why sort by ratio in B&B;?

A. It makes the fractional upper bound useful.

Q. Does B&B; final answer include fractions?

A. No. Fractions are used only in the optimistic bound.

Q. Why use selected.copy()?

A. To save an independent snapshot of the best selected items.

Q. Why can B&B; still be exponential?

A. In the worst case, the bound may not prune enough branches.

Q. Should DP and B&B; return the same profit?

A. Yes. Both solve the same exact optimization problem for the same valid input.

5. N-Queens with the First Queen Fixed

File: NQueens.py

Problem statement

Place n queens on an n×n chessboard so that no two queens attack each other. The first queen is already placed at a row and column entered by the user. Use backtracking to place the remaining queens and print a final board if one exists.

Theory

A queen attacks along its row, column, and diagonals. A valid arrangement has no two queens sharing any of these lines. The solver places one queen in each remaining row, reducing the search space.

How the code works

• The board is an n×n list: 1 means queen; 0 means empty.

• The user enters the fixed queen's row and column (1-based); the program validates and places it.

• The row containing the fixed queen is skipped; remaining rows are processed in order.

• For each row, try every column and call is_safe before placing a queen.

• is_safe checks the candidate row and column, then scans four diagonal directions: up-left, up-right, down-left, down-right.

• After a safe placement, print the intermediate board and recurse to the next row.

• If a later row cannot be completed, reset the previous cell to 0, print the backtrack state, and try another column.

• When every remaining row has a queen, return True. This implementation stops after the first solution.

Why four diagonal directions?

An attacking queen may be on either diagonal slope and on either side of the candidate square. The direction pairs are (–1,–1), (–1,+1), (+1,–1), and (+1,+1).

Complexity

The row-by-row search has a factorial-style bound, commonly described as O(n!) for the placement search. Each safety check scans up to O(n) squares, so a conservative overall time bound is O(n·n!). The board takes O(n²) space and recursion stack O(n).

Fixed position matters

The solver completes the board while keeping the given first queen fixed. Not every fixed position permits a solution. For n=4, fixing the queen at row 1, column 2 permits a solution; fixing it at row 1, column 1 does not.

```
n = 4, fixed queen at (row 1, column 2)\nOne valid board:\n0 1 0 0\n0 0 0 1\n1 0 0 0\n0 0 1 0
```

Test checks

• n=1 at (1,1): solution.

• n=2 or n=3: no solution exists.

• n=4 at (1,2): a solution exists.

• n=4 at (1,1): no solution with that fixed queen.

• A row or column outside 1..n should be rejected.

Likely viva questions

Q. Why is this backtracking?

A. A placement can block later rows; when that happens, undo it and try another column.

Q. What does is_safe check?

A. Same row, same column, and all four diagonal directions.

Q. Why use a 2D matrix?

A. It represents the chessboard directly and makes placement and printing easy.

Q. Why is the first queen fixed?

A. It is given by the problem; the solver must complete around it.

Q. Does every fixed position lead to a solution?

A. No. Some fixed positions make completion impossible.

Q. Does the program find all solutions?

A. No, it stops at the first valid solution.

Q. What happens if a row has no safe column?

A. The function returns False so the previous call can undo its queen and try another choice.

Q. Why reset a cell to zero?

A. To undo a tentative placement before trying another option.

Q. Why are n=2 and n=3 impossible?

A. No arrangement of 2 or 3 queens on those board sizes avoids all attacks.

Q. What is the board space complexity?

A. $O(n^2)$, plus $O(n)$ recursion stack.

6. Cross-Practical Comparisons

Topic	Technique	Central idea
Fibonacci	Recursion / iteration	Compare direct recurrence with a loop.
Tower of Hanoi	Recursion	Two smaller transfers around one largest-disk move.
Huffman	Greedy + min-heap	Combine the two least-frequent nodes.
Fractional Knapsack	Greedy + sorting	Highest value per unit weight first.
0/1 Knapsack DP	Dynamic Programming	Reuse best values for smaller subproblems.
0/1 Knapsack B&B	Branch and Bound	Explore include/exclude choices; prune weak branches.
N-Queens	Backtracking	Try safe placements; undo choices when stuck.

Common examiner questions

Q. Algorithm vs program?

A. An algorithm is a finite step-by-step method; a program implements it in a language.

Q. Why analyze complexity instead of only measuring time?

A. Measurements depend on the machine; complexity describes growth with input size.

Q. What is worst-case complexity?

A. An upper bound on resources needed for the most demanding input of a given size.

Q. Which practical uses a min-heap?

A. Huffman Coding.

Q. Which practicals use greedy methods?

A. Huffman Coding and Fractional Knapsack.

Q. Which practical uses a DP table?

A. 0/1 Knapsack using Dynamic Programming.

Q. Which practical uses pruning with a bound?

A. 0/1 Knapsack using Branch and Bound.

Q. Which practical fixes a starting queen?

A. N-Queens.

Q. Greedy vs backtracking?

A. Greedy commits to local choices; backtracking can undo a choice when it leads to a dead end.

Q. DP vs B&B;?

A. DP stores subproblem optima; B&B; searches a decision tree and prunes using bounds.

7. Linux Execution and Test Checklist

Check Python 3

```
python3 --version
```

On Ubuntu/Debian, if Python 3 is missing:

```
sudo apt update\nsudo apt install python3
```

Open the project directory

```
cd /path/to/your/DAA-folder\nls
```

Check that all six .py files are present. Linux filenames are case-sensitive.

Run the programs

```
python3 Fibonacci_Hanoi.py\npython3 Huffman_Coding.py\npython3 Fractional_Knapsack_Greedy.py\npython3 01Knapsack_
```

No external packages are needed; the programs use Python's standard library. Huffman imports heapq, which is included with Python.

Quick test matrix

Program	Input	Expected check
Fibonacci	7 terms	0 1 1 2 3 5 8
Hanoi	3 disks	7 moves
Huffman	AAABBC	Decode(Encode(text)) equals original
Huffman single symbol	AAAAA	Code 0; encoded 00000
Fractional Knapsack	Values 60,100,120; weights 10,20,30; cap 50	Maximum value 240
0/1 DP	Same items; cap 50	Maximum value 220
0/1 B&B	Same items; cap 50	Maximum value 220
N-Queens	n=4; fixed queen (1,2)	A valid arrangement is found

Testing tips

• Test a normal input, then a boundary input and an invalid input if validation exists.

• Run identical 0/1 Knapsack data through DP and B&B; optimal profits should match.

• For Huffman, compare decoded output with the original string, not one memorized bit pattern.

• For N-Queens, the fixed starting square can change whether a solution exists.

8. Last-Minute Viva Sheet

Question	Short answer
Fibonacci recurrence?	F(n)=F(n–1)+F(n–2), F(0)=0, F(1)=1.
Iterative Fibonacci?	O(n) time, O(1) auxiliary space.
Hanoi moves?	2^n–1.
Huffman method?	Greedily combine two minimum-frequency nodes using a min-heap.
Prefix-free code?	No codeword is a prefix of another codeword.
Fractional Knapsack choice?	Sort by descending value/weight ratio.
Fractional Knapsack time?	O(n log n).
0/1 Knapsack?	Each item is either fully taken or not taken.
DP state?	dp[i][w] = best value using first i items and capacity w.
0/1 DP complexity?	O(nW) time and O(nW) table space.
Branch and Bound?	Search choices; prune branches whose upper bound cannot beat the incumbent.
N-Queens safety?	No other queen in the same row, column, or diagonal.
Backtracking?	Try a choice; if it fails, undo it and try another.

30-second explanations

Q. Fibonacci and Hanoi

A. The program generates Fibonacci terms recursively and iteratively, measures elapsed time, and solves Hanoi recursively. Recursive Fibonacci repeats work; iteration stores two running values. Hanoi makes two n–1 transfers around one largest-disk move, so it needs 2^n–1 moves.

Q. Huffman

A. The program counts character frequencies, builds a tree by repeatedly combining the two least-frequent nodes from a min-heap, generates prefix-free codes, and offers encoding and decoding in a menu.

Q. Fractional Knapsack

A. The program sorts items by value/weight ratio, takes best-ratio items fully while possible, and takes a fraction of the next item. Greedy is optimal because items are divisible.

Q. 0/1 DP

A. The program fills a two-dimensional table comparing include and exclude choices for each item and capacity, then backtracks to report selected whole items.

Q. 0/1 Branch and Bound

A. The program explores include/exclude choices after ratio sorting. A fractional relaxation gives an optimistic upper bound; branches unable to beat the best profit are pruned. Final items are whole items only.

Q. N-Queens

A. The program keeps the first queen fixed and uses backtracking to place one queen in each remaining row. It checks columns and diagonals, prints intermediate placements, and removes a queen when a later choice fails.

Final advice: explain the idea before individual lines. Trace one small example by hand. Know each function's inputs and outputs, the role of its data structure, why its conditions are present, and what its complexity means. If asked about a limitation, state it honestly.